

CIS022 DAY 9

File reading and input validation

PART 1

Reading and writing files

FILE READING

Here is a short program that prints every line of a file named `pg2383.txt`

(it's the Project Gutenberg version of Chaucer's Canterbury Tales, if you're interested)

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main() {
8     ifstream inputFile("pg2383.txt");
9
10    if (!inputFile) {
11        cerr << "Could not open file!" << endl;
12        return 1;
13    }
14
15    string line;
16    while (getline(inputFile, line)) {
17        cout << line << '\n';
18    }
19    return 0;
20 }
```

FILE READING

key things to notice:

To work with files, include the
<fstream> library



```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main() {
8     ifstream inputFile("pg2383.txt");
9
10    if (!inputFile) {
11        cerr << "Could not open file!" << endl;
12        return 1;
13    }
14
15    string line;
16    while (getline(inputFile, line)) {
17        cout << line << '\n';
18    }
19    return 0;
20 }
```

FILE READING

key things to notice:

To work with files, include the `<fstream>` library

Declare an `ifstream` (input file stream) and give it the file name. This opens the file.



```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main() {
8     ifstream inputFile("pg2383.txt");
9
10    if (!inputFile) {
11        cerr << "Could not open file!" << endl;
12        return 1;
13    }
14
15    string line;
16    while (getline(inputFile, line)) {
17        cout << line << '\n';
18    }
19    return 0;
20 }
```

FILE READING

key things to notice:

If the file didn't open, inputFile counts as false. Always check!



```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main() {
8     ifstream inputFile("pg2383.txt");
9
10    if (!inputFile) {
11        cerr << "Could not open file!" << endl;
12        return 1;
13    }
14
15    string line;
16    while (getline(inputFile, line)) {
17        cout << line << '\n';
18    }
19    return 0;
20 }
```

FILE READING

key things to notice:

cerr is like cout, but meant for error messages. Returning 1 from main tells the system something went wrong.



```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main() {
8     ifstream inputFile("pg2383.txt");
9
10    if (!inputFile) {
11        cerr << "Could not open file!" << endl;
12        return 1;
13    }
14
15    string line;
16    while (getline(inputFile, line)) {
17        cout << line << '\n';
18    }
19    return 0;
20 }
```

FILE READING

key things to notice:

`getline(stream, line)` reads everything up to the next newline into `line`. The newline itself is thrown away.



```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main() {
8     ifstream inputFile("pg2383.txt");
9
10    if (!inputFile) {
11        cerr << "Could not open file!" << endl;
12        return 1;
13    }
14
15    string line;
16    while (getline(inputFile, line)) {
17        cout << line << '\n';
18    }
19    return 0;
20 }
```

FILE READING

key things to notice:

The loop condition is the read itself.
When `getline()` runs out of file, it
fails, and the loop ends.



```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main() {
8     ifstream inputFile("pg2383.txt");
9
10    if (!inputFile) {
11        cerr << "Could not open file!" << endl;
12        return 1;
13    }
14
15    string line;
16    while (getline(inputFile, line)) {
17        cout << line << '\n';
18    }
19    return 0;
20 }
```

FILE READING

key things to notice:

No `close()` needed: the file closes itself when `inputFile` goes away at the end of `main`. (More on this in a moment.)



```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main() {
8     ifstream inputFile("pg2383.txt");
9
10    if (!inputFile) {
11        cerr << "Could not open file!" << endl;
12        return 1;
13    }
14
15    string line;
16    while (getline(inputFile, line)) {
17        cout << line << '\n';
18    }
19    return 0;
20 }
```

WHY WON'T MY FILE OPEN?

The #1 reason: the file isn't where your program is looking.

A plain name like `pg2383.txt` is looked for in the program's working directory, usually the folder it was run from. IDEs often run your program from a different folder than the one your `.cpp` file is in.

Fixes: move the file there, or use the full path, like
`/Users/me/cis25/pg2383.txt`

Also check the spelling, and watch for hidden extensions:
`pg2383.txt.txt` is a classic.

Output

```
$ ./read  
Could not open file!
```

DO I NEED close()?

When a stream variable goes out of scope, its file is closed for you



```
1 void printFirstLine() {  
2     ifstream inputFile("pg2383.txt");  
3     string line;  
4     getline(inputFile, line);  
5     cout << line << '\n';  
6 } // inputFile is closed here, automatically
```

This pattern has a name: RAI (Resource Acquisition Is Initialization). We'll see how it works when we write our own classes.

Call `close()` yourself only if you're finished with a file early, or want to reuse the stream for a different file.

DON'T LOOP ON eof()

`eof()` only turns true after a read has already failed. So this loop runs one extra time, and writes an extra blank line at the end.



what you'll often see online

```
16     while (!inputFile.eof()) {  
17         getline(inputFile, line);  
18         outputFile << line << '\n';  
19     }
```

`getline()` gives back the stream, which counts as false once a read fails. The loop stops exactly when it should.



what to write instead

```
16     while (getline(inputFile, line)) {  
17         outputFile << line << '\n';  
18     }
```

FILE WRITING

Writing works a lot like reading.
This program copies pg2383.txt
into output.txt.

(same #includes as before)

```
7 int main() {
8     ifstream inputFile("pg2383.txt");
9     ofstream outputFile("output.txt");
10
11     if (!inputFile || !outputFile) {
12         cerr << "Could not open file!" << endl;
13         return 1;
14     }
15     string line;
16     while (getline(inputFile, line)) {
17         outputFile << line << '\n';
18     }
19     return 0;
20 }
```

FILE WRITING

key things to notice:

An ofstream (output file stream) opens a file for writing. If the file doesn't exist, it's created. If it does, it's erased first!



```
7 int main() {
8     ifstream inputFile("pg2383.txt");
9     ofstream outputFile("output.txt");
10
11     if (!inputFile || !outputFile) {
12         cerr << "Could not open file!" << endl;
13         return 1;
14     }
15     string line;
16     while (getline(inputFile, line)) {
17         outputFile << line << '\n';
18     }
19     return 0;
20 }
```

FILE WRITING

key things to notice:

Write with <<, exactly like you
would with cout



```
7 int main() {
8     ifstream inputFile("pg2383.txt");
9     ofstream outputFile("output.txt");
10
11     if (!inputFile || !outputFile) {
12         cerr << "Could not open file!" << endl;
13         return 1;
14     }
15     string line;
16     while (getline(inputFile, line)) {
17         outputFile << line << '\n';
18     }
19     return 0;
20 }
```

'\n' VS endl

```
outputFile << line << endl;    // newline, then flush  
outputFile << line << '\n';    // just the newline
```

Output is buffered: C++ saves up what you write and sends it to the file in big batches.

endl forces the batch out every single line. For a file with thousands of lines, that's thousands of separate writes instead of a few big ones.

Use '\n' when writing lots of lines. endl is fine for the occasional line on the console.

ADDING TO A FILE

`ios::app` means append: add to the end of the file instead of erasing it first



```
1 #include <fstream>
2
3 using namespace std;
4
5 int main() {
6     ofstream logFile("log.txt", ios::app);
7     logFile << "Program ran!" << '\n';
8     return 0;
9 }
```

Two runs, two lines. Without `ios::app`, you'd only ever see one.



Output (cat prints a file's contents)

```
$ ./logger
$ ./logger
$ cat log.txt
Program ran!
Program ran!
```

PASSING STREAMS TO FUNCTIONS

Always pass streams by reference (&).
Streams can't be copied, so passing one by value won't even compile.



```
7 void printFiveWords(ifstream& inputFile) {  
8     string word;  
9     for (int i = 0; i < 5; i++) {  
10         inputFile >> word;  
11         cout << word;  
12     }  
13     cout << endl;  
14 }
```

PASSING STREAMS TO FUNCTIONS

Reading moves the stream forward, for the caller too. The next read picks up wherever this one left off.



```
7 void printFiveWords(ifstream& inputFile) {  
8     string word;  
9     for (int i = 0; i < 5; i++) {  
10         inputFile >> word;  
11         cout << word;  
12     }  
13     cout << endl;  
14 }
```

>> SKIPS WHITESPACE

>> reads one word at a time. It skips whitespace, and doesn't keep it.



```
7 void printFiveWords(ifstream& inputFile) {  
8     string word;  
9     for (int i = 0; i < 5; i++) {  
10         inputFile >> word;  
11         cout << word;  
12     }  
13     cout << endl;  
14 }
```

first line of pg2383.txt

```
WHANNE that Aprille, with his shoures soote
```

The spaces are gone. That's why `getline()` is usually the better tool for files.



Output

```
$ ./words  
WHANNEthatAprille,withhis
```

THE NEWLINE TRAP

```
7    int age;  
8    string name;  
9    cout << "Age: ";  
10   cin >> age;  
11   cout << "Name: ";  
12   getline(cin, name);  
13   cout << "Hi, " << name << "!" << endl;
```

Output

```
$ ./trap  
Age: 20  
Name: Hi, !
```

It never waits for a name!



THE NEWLINE TRAP

`cin >> age` reads the 20, but leaves the newline behind. `getline()` finds it right away and returns an empty line.



```
7    int age;  
8    string name;  
9    cout << "Age: ";  
10   cin >> age;  
11   cout << "Name: ";  
12   getline(cin, name);  
13   cout << "Hi, " << name << "!" << endl;
```

Output

```
$ ./trap  
Age: 20  
Name: Hi, !
```

what's in `cin` after `cin >> age`:

2	0	\n
---	---	----



next character to read

STREAM STATE: ONE IDEA

What happened	Flags that get set	if (stream) is...
The file didn't open	failbit	false
getline() ran out of file	eofbit + failbit	false
Letters where a number was expected	failbit	false

this is why `if (inputFile)` and `while (getline(...))` work



Three different problems, one mechanism. Once failbit is set, every read fails immediately, until you call `clear()`.

CHECKING THE FLAGS

You'll need this one in part 2



```
if (stream)           // false if a failure flag is set
stream.fail()         // true if a failure flag is set
stream.eof()          // true if a read hit the end
stream.clear()        // resets all the flags
```

`if (stream)` and `stream.fail()` always give opposite answers. Use whichever reads better.

There's also a `badbit`, for serious problems like a disk error. You'll rarely need to think about it.

PART 1 RECAP

Open files by declaring an `ifstream` or `ofstream`, then check with `if (file)`

Loop on the read itself: `while (getline(file, line))`

Streams close their files automatically when they go out of scope (RAII)

`>>` reads a word at a time and skips whitespace; `getline()` reads a whole line

Every stream keeps track of what went wrong using flags

PART 2

Handling bad input

ERROR HANDLING: the fail flag

Consider this
(relatively useless)
program

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     int x = 0;
7     int y = 0;
8     cout << "Enter x: ";
9     cin >> x;
10    cout << "Enter y: ";
11    cin >> y;
12    cout << "Your point is (" << x << ", " << y << ")" << endl;
13    return 0;
14 }
```

Output

```
$ ./point
Enter x: 2
Enter y: 3
Your point is (2, 3)
```

Sensible input,
sensible result



ERROR HANDLING: the fail flag

Consider this
(relatively useless)
program

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     int x = 0;
7     int y = 0;
8     cout << "Enter x: ";
9     cin >> x;
10    cout << "Enter y: ";
11    cin >> y;
12    cout << "Your point is (" << x << ", " << y << ")" << endl;
13    return 0;
14 }
```

Output

```
$ ./point
Enter x: a
Enter y: Your point is (0, 0)
```

It never even
waited for y!



WHAT HAPPENED?

`cin >> x` found an `a` where it needed a number. So it set failbit (and stored 0 in `x`).

While failbit is set, every read fails instantly, without waiting for input. That's why `y` was never read.

And the `a` is still sitting in the stream.

what's in cin:



next character to read

clear() ISN'T ENOUGH

```
cin >> x;           // fails on the "a"  
cin.clear();         // failbit is reset...  
cin >> y;           // ...but the "a" is still there, so this fails too
```

what's in cin after clear():



next character to read

clear() resets the flags, but it doesn't remove anything from the stream. We also need a way to throw the bad input away.

THROWING IT AWAY: ignore()

```
cin.ignore(1000, '\n');
```

ignore(count, stopChar) throws away up to count characters, stopping once it has thrown away stopChar.

In practice: a big number and '\n', which means "throw away the rest of this line."

```
#include <limits>
...
cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

If you want to be thorough: that's the largest count possible, so it always reaches the newline. A plain big number is fine for this class.

CLEAR, THEN IGNORE

When a read fails, reset the flags and throw away the bad line



```
9      cout << "Enter x: ";
10     cin >> x;
11     if (cin.fail()) {
12         cin.clear();
13         cin.ignore(numeric_limits<streamsize>::max(), '\n');
14     }
15     cout << "Enter y: ";
16     cin >> y;
17     cout << "Your point is (" << x << ", " << y << ")" << endl;
```

Output

Still no x, but at least y works now



```
$ ./point
Enter x: a
Enter y: 3
Your point is (0, 3)
```

KEEP ASKING UNTIL IT WORKS

If the read fails,
clean up and try
again



```
7   int x;  
8   cout << "Enter x: ";  
9   while (!(cin >> x)) {  
10      cin.clear();  
11      cin.ignore(numeric_limits<streamsize>::max(), '\n');  
12      cout << "That's not a number. Try again: ";  
13  }  
14  cout << "x is " << x << endl;
```

Output

Bad input no
longer breaks the
program



```
$ ./retry  
Enter x: abc  
That's not a number. Try again: two  
That's not a number. Try again: 7  
x is 7
```

BUT >> IS STILL FRAGILE

No error at all: >> reads as much as it can, and stops at the first character that doesn't fit



Output

```
$ ./retry  
Enter x: 12abc  
x is 12
```

Same story: 3.7 quietly becomes 3



Output

```
$ ./retry  
Enter x: 3.7  
x is 3
```

The leftovers (abc, .7) stay in cin and will trip up the next read. >> can't tell us whether the whole line was a number.

READ THE WHOLE LINE FIRST

```
string line;  
getline(cin, line);    // accepts anything they type
```

`getline()` accepts any text, so `cin` never ends up in a failed state. No `clear()`, no `ignore()`.

It also avoids the newline trap from part 1, since we never mix `>>` and `getline()` on `cin`.

The only question left: is this string a valid number?

READING FROM A STRING

An `istringstream` is a stream that reads from a string. It needs `#include <sstream>`



```
8   istringstream in("42 is the answer");
9   int n;
10  string word;
11  in >> n >> word;
12  cout << n << " " << word << endl;
```

Everything you know about streams works here: `>>`, `getline()`, `if (in)`



Output

```
$ ./sstream
42 is
```

CHECKING A STRING: parseInt()

Let's wrap that up in a function we can reuse

```
7 bool parseInt(string text, int& value) {  
8     stringstream in(text);  
9     if (!(in >> value)) {  
10         return false;    // no number at the start  
11     }  
12     char extra;  
13     if (in >> extra) {  
14         return false;    // leftovers, like "12abc"  
15     }  
16     return true;  
17 }
```

CHECKING A STRING: parseInt()

Returns whether it worked. The number itself comes back through `value`, which is passed by reference.



```
7 bool parseInt(string text, int& value) {  
8     stringstream in(text);  
9     if (!(in >> value)) {  
10         return false;    // no number at the start  
11     }  
12     char extra;  
13     if (in >> extra) {  
14         return false;    // leftovers, like "12abc"  
15     }  
16     return true;  
17 }
```

CHECKING A STRING: parseInt()

First, try to read a number from the start of the string



Then try to read one more character. If that works, there was junk after the number.



```
7 bool parseInt(string text, int& value) {  
8     istream in(text);  
9     if (!(in >> value)) {  
10         return false;    // no number at the start  
11     }  
12     char extra;  
13     if (in >> extra) {  
14         return false;    // leftovers, like "12abc"  
15     }  
16     return true;  
17 }
```

>> skips whitespace, so " 7 " still counts as a number.

USING parseInt()

Keep reading lines until one of them is a valid number



```
19 int main() {  
20     int x;  
21     string line;  
22     cout << "Enter x: ";  
23     while (getline(cin, line) && !parseInt(line, x)) {  
24         cout << "That's not a number. Try again: ";  
25     }  
26     cout << "x is " << x << endl;  
27     return 0;  
28 }
```

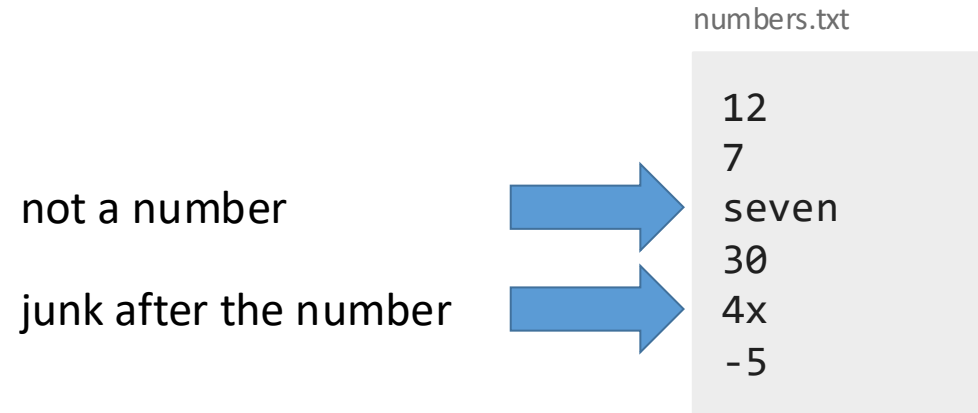
All three bad inputs are rejected. No clear(), no ignore().



Output

```
$ ./parse  
Enter x: abc  
That's not a number. Try again: 12abc  
That's not a number. Try again: 3.7  
That's not a number. Try again: 7  
x is 7
```

WORKED EXAMPLE



Task: add up the numbers in `numbers.txt`. Skip the bad lines, and report which line they were on.

We already have everything we need: the `getline()` loop from part 1, and `parseInt()`. It doesn't care whether a line came from `cin` or from a file.

WORKED EXAMPLE

(opening and checking the file: same as always)

Same loop as always



Same parseInt() as before



Report problems on cerr



```
26 string line;
27 int lineNumber = 0;
28 int total = 0;
29 while (getline(inputFile, line)) {
30     lineNumber++;
31     int value;
32     if (parseInt(line, value)) {
33         total += value;
34     } else {
35         cerr << "Skipping line " << lineNumber << '\n';
36     }
37 }
38 cout << "Total: " << total << endl;
```

WORKED EXAMPLE

```
26 string line;
27 int lineNumber = 0;
28 int total = 0;
29 while (getline(inputFile, line)) {
30     lineNumber++;
31     int value;
32     if (parseInt(line, value)) {
33         total += value;
34     } else {
35         cerr << "Skipping line " << lineNumber << '\n';
36     }
37 }
38 cout << "Total: " << total << endl;
```

Output

```
$ ./sum
Skipping line 3
Skipping line 5
Total: 44
```

$$12 + 7 + 30 + (-5) = 44$$



SUMMARY

Open files with `ifstream / ofstream`, and check them with `if (file)`

Loop on the read itself: `while (getline(file, line))`

A failed read sets failbit. `clear()` resets the flags; `ignore()` throws away the bad input.

Most robust: read whole lines with `getline()`, then check them with `parseInt()`

Streams close their files automatically (RAII)

SNEAK PEEK

Exceptions (next time)

stoi(): STRING TO INT

stoi() ("string to int") turns a string into an int



```
7   string line;  
8   getline(cin, line);  
9   int n = stoi(line);  
10  cout << "n is " << n << endl;
```

Output

```
$ ./stoi  
abc  
terminate called after throwing an instance of 'std::invalid_argument'  
    what():  stoi  
Aborted
```

On bad input, stoi() doesn't set a flag. It throws an exception, and if nothing catches it, the program ends.

stoi(): STRING TO INT

```
7    string line;  
8    getline(cin, line);  
9    int n = stoi(line);  
10   cout << "n is " << n << endl;
```

And it quietly accepts 12abc. We'd still need our leftovers check.

Output



```
$ ./stoi  
12abc  
n is 12
```

WHY EXCEPTIONS?

With flags, every piece of code has to remember to check. It's easy to forget, and then things fail silently (like our point program).

An exception can't be silently ignored. It travels up until some code catches it, ideally code that knows what to do about the problem.

And thanks to RAII, open files still get closed along the way.

next time:

```
try {  
    int n = stoi(line);  
    cout << "n is " << n << endl;  
} catch (const invalid_argument& e) {  
    cout << "That's not a number!" << endl;  
}
```